

UNIDADE I

Garantia de Qualidade de Software (Parte II)



Profa. Ana Carolina Gondim Inocêncio

Garantia de Qualidade de Software — Parte I

- ✓ SQA é uma atividade universal — não começa depois que o código é gerado.
- ✓ Vimos as nove frentes de atuação da SQA: padrões, revisões e auditorias, testes, mudanças, educação, fornecedores, segurança, riscos.
- ✓ O grupo de SQA planeja, revisa, audita e acompanha desvios até o fechamento.
- ✓ Fechamos vendo que boa parte desse trabalho hoje está automatizada em *quality gates* de CI/CD.

Hoje: como medir se a qualidade está sendo alcançada — metas, métricas, estatística e confiabilidade.

Garantia de Qualidade de Software — Parte II

- 1 Metas, atributos e métricas da SQA
- 2 Estatísticas da Garantia da Qualidade de Software
- 3 Confiabilidade de Software
- 4 O Plano de SQA
- 5 SQA na prática moderna — SRE, SLO e error *budgets*
- 6 Kahoot valendo nota (AC1)

Na aula passada vimos os *quality gates* automatizados. Mas como saber SE aquilo que automatizamos está realmente funcionando bem?

PollEveryWhere – Aula 06

Ligando com a Aula 5

- Automatizar um *gate* de qualidade não basta — é preciso medir se ele está cumprindo seu papel.
- Hoje vamos ver como a SQA transforma qualidade em algo mensurável: metas, métricas, estatística e confiabilidade.
- No fim, vamos ver que essas mesmas fórmulas viraram a base da disciplina de *Site Reliability Engineering (SRE)*.



01

Metas, Atributos e Métricas da SQA



Metas Pragmáticas

As ações de garantia da qualidade são realizadas para atingir um conjunto de metas pragmáticas — objetivas, concretas, verificáveis.

Duas Primeiras Metas

Qualidade dos Requisitos

A correção, completude e consistência do modelo de requisitos influenciam fortemente a qualidade de todos os produtos seguintes.

Qualidade do Projeto

Todo elemento do modelo de projeto deve ser avaliado pela equipe, garantindo alta qualidade e conformidade com os requisitos.

Mais Duas Metas

Qualidade do Código

O código-fonte e produtos relacionados devem seguir os padrões locais de codificação e facilitar a manutenção.

Eficácia do Controle de Qualidade

A equipe aplica recursos limitados da forma mais eficaz possível; a SQA analisa se revisões e testes estão bem alocados.

De Meta a Número: o Caminho

Meta

O que se busca — ex.: alta qualidade de requisitos.

Atributo

Uma característica observável associada à meta — ex.: ambiguidade de um requisito.

Métrica

Uma medida concreta desse atributo — ex.: número de termos ambíguos por página de especificação.

POLL EVERYWHERE · RESPONDAM PELO CELULAR

Das quatro metas pragmáticas que vimos, qual vocês acham mais difícil de medir na prática?

PollEveryWhere

Vamos projetar as respostas na tela em tempo real — sem certo ou errado ainda.

O Que Não Se Mede, Não Se Gerencia

- Requisitos e projeto costumam ser os mais difíceis — depende de julgamento humano, não só de contagem automática.
- Código e controle de qualidade têm métricas mais diretas (cobertura, densidade de defeitos), por isso são mais fáceis de automatizar.
- Isso não os torna mais importantes — só mais fáceis de vigiar.



02

Estatísticas da Garantia da Qualidade de Software

Uma Análise Mais Quantitativa

A estatística da garantia de qualidade reflete uma tendência crescente em toda a indústria de software para tornar mais quantitativa a análise da qualidade.

Como Funciona, Passo a Passo

- Informações sobre erros de software são coletadas e classificadas
- É feita uma tentativa de associar cada erro à sua causa subjacente
- Usando o Princípio de Pareto, isolam-se as causas vitais (20% das causas)
- Prossegue-se para a correção dos problemas que provocaram os defeitos

O Princípio de Pareto

80% dos defeitos podem ser associados a 20% de todas as possíveis causas.

A mesma regra 80/20 que aparece em vendas, trânsito e uso de recursos — aqui aplicada a defeitos de software.

Estatística de SQA e Pareto, em Uma Frase

Esse conceito é um passo importante para uma gestão de qualidade adaptativa, em que mudanças melhoram os elementos do processo que mais introduzem erros.

“Invista seu tempo concentrando-se em coisas que realmente importam — mas primeiro, certifique-se de ter entendido aquilo que realmente importa.”

POLL EVERYWHERE · RESPONDAM PELO CELULAR

Nos projetos de vocês, que tipo de causa vocês diriam que está entre os 20% vitais dos defeitos?

PollEveryWhere

Seis Sigma para Engenharia de Software

A estratégia para estatística da garantia da qualidade mais utilizada na indústria: uma metodologia rigorosa e disciplinada que usa análise estatística para identificar e eliminar defeitos em processos.

O Método DMAAC

Definir

As necessidades do cliente e os artefatos passíveis de entrega.

Medir

O processo existente e seu resultado, para determinar o desempenho de qualidade atual.

Analisar

As métricas de defeitos, para determinar as poucas causas vitais.

Aperfeiçoar

Eliminar as causas fundamentais dos defeitos identificados.

Controlar

Garantir que trabalhos futuros não reintroduzam essas causas.



03

Confiabilidade de Software

O Que É Confiabilidade?

Definida em termos estatísticos como “a probabilidade de operação sem falhas de um programa de computador em um dado ambiente, por um determinado tempo”.

Pode ser medida diretamente ou estimada com dados históricos e de desenvolvimento.

MTBF: Tempo Médio Entre Falhas

Uma medida simples de confiabilidade é o tempo médio entre falhas de um sistema computacional.

$$MTBF = MTTF + MTTR$$

- *MTTF = tempo médio para falhar*
- *MTTR = tempo médio para reparar*

As Duas Armadilhas do MTBF

- Projeta um período de tempo entre falhas, mas não fornece uma projeção da taxa de falhas
- Pode ser mal interpretado como “tempo de vida médio” — não é esse o seu significado

Uma Medida Alternativa: FIT

Failures-in-time (FIT): uma medida estatística de quantas falhas um componente terá ao longo de um bilhão de horas de operação.

1 FIT = uma falha a cada bilhão de horas de operação.

Disponibilidade

Disponibilidade de software é a probabilidade de que um programa esteja operando de acordo com os requisitos em um dado instante.

$$\textit{Disponibilidade} = [MTTF / (MTTF + MTTR)] \times 100\%$$

Confiabilidade × Proteção

Confiabilidade

- Análise estatística para determinar a probabilidade de ocorrência de uma falha de software.

Proteção (Safety)

- Examina as maneiras pelas quais falhas resultam em condições que podem levar a problemas maiores.

Proteção de Software — Um Exemplo

- Provocar uma aceleração descontrolada que não pode ser interrompida
- Não responder ao acionamento do pedal do freio
- Não operar quando a chave é ativada
- Perder ou ganhar velocidade lentamente, sem comando

Confiabilidade e Proteção Não São a Mesma Coisa

- Confiabilidade pergunta: “com que probabilidade isso vai falhar?”
- Proteção pergunta: “se isso falhar, o que de pior pode acontecer — e como evitamos?”
- Um sistema pode ser estatisticamente confiável e ainda assim ter um risco de proteção inaceitável — por isso as duas análises são feitas separadamente.



04

O Plano de SQA

O Roteiro da Garantia da Qualidade

O plano de SQA fornece um roteiro para instituir a garantia da qualidade de software. Desenvolvido pelo grupo de SQA, serve como gabarito para as atividades de SQA de cada projeto.

A Estrutura IEEE do Plano de SQA

- Propósito e escopo do plano
- Artefatos de ES sob o escopo da SQA (modelos, documentos, código)
- Padrões e práticas aplicados na gestão de qualidade
- Ações e tarefas da SQA (revisões, auditorias)
- Ferramentas e métodos de apoio às tarefas de SQA
- Procedimentos de gestão de configuração de software
- Métodos de registro e manutenção dos registros de SQA
- Papéis e responsabilidades relacionados à qualidade

O Plano Não É Burocracia — É Memória Institucional

- Sem um plano escrito, cada projeto reinventa suas próprias regras de qualidade — e perde o que funcionou no projeto anterior.
- Os itens mais difíceis costumam ser papéis/responsabilidades e ferramentas — porque dependem de decisões organizacionais, não só técnicas.



05

SQA na Prática Moderna

O Mesmo MTBF, Nome Novo

As fórmulas de confiabilidade e disponibilidade que vimos são exatamente a base matemática da disciplina de *Site Reliability Engineering* (SRE), popularizada pelo Google.

MTTF e MTTR viraram vocabulário comum em qualquer time de produção.

SLI, SLO, SLA e *Error Budget*

SLI

Service Level Indicator — uma métrica observável, ex.: % de requisições bem-sucedidas.

SLO

Service Level Objective — a meta interna de confiabilidade que o time se compromete a proteger.

SLA

Service Level Agreement — o contrato externo, com penalidades se a meta não for cumprida.

Error Budget (orçamento de erros, de forma literal)

A margem de falha permitida: 100% menos o SLO.

Error Budget na Prática

Se a meta (SLO) é 99,9% de disponibilidade, o error budget é 0,1% — a mesma disponibilidade que calculamos com $MTTF/(MTTF+MTTR)$, só que orçada com antecedência.

Orçamento esgotado antes do fim do período? A prática de SRE recomenda congelar novas funcionalidades até restaurar a confiabilidade.

O Que É um Error Budget?

Pense num orçamento financeiro do mês: se sua meta é gastar no máximo R\$1.000, sobra uma margem de imprevistos. O error budget é a mesma ideia, só que a “moeda” é indisponibilidade: se a meta (SLO) é 99,9%, a “margem” que você pode gastar falhando é 0,1%.

$$\text{Error Budget} = 100\% - \text{SLO}$$

Na prática, quanto tempo é essa “margem” por mês? Depende de quantos “noves” tem a sua meta:

SLO 99%	budget 1%	≈ 7h12min de indisponibilidade por mês
SLO 99,9%	budget 0,1%	≈ 43 minutos de indisponibilidade por mês
SLO 99,99%	budget 0,01%	≈ 4 minutos e 19s de indisponibilidade por mês
SLO 99,999%	budget 0,001%	≈ 26 segundos de indisponibilidade por mês

Como Calculamos os Minutos da Tabela?

A tabela do slide anterior não foi feita no chute — cada valor vem da mesma fórmula, aplicada a um mês de referência:

$$\text{Indisponibilidade permitida} = (100\% - \text{SLO}) \times \text{duração do período}$$

Exemplo passo a passo, para SLO = 99,9%:

1. Mês de referência $30 \text{ dias} \times 24\text{h} \times 60\text{min} = 43.200 \text{ minutos no total.}$

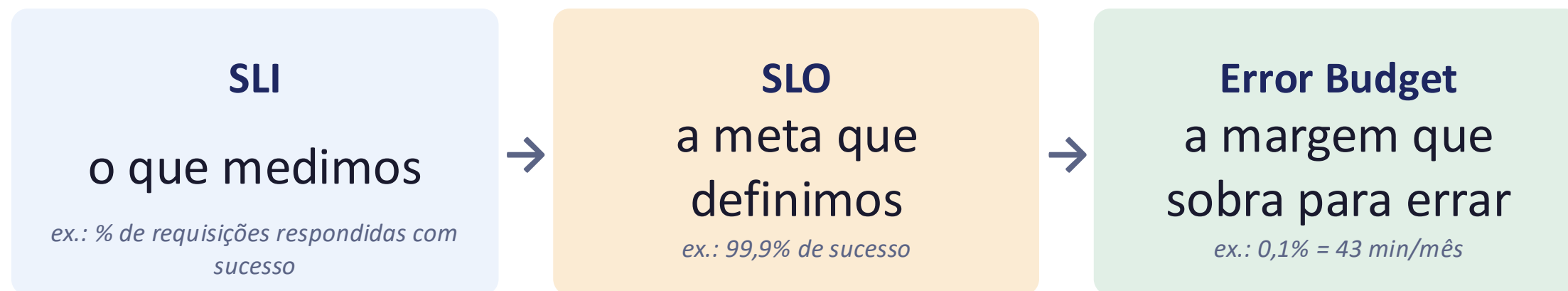
2. Margem de erro $100\% - 99,9\% = 0,1\%$ (a fatia que pode falhar).

3. Indisponibilidade permitida $0,1\% \times 43.200 = 43,2 \text{ minutos} \approx 43 \text{ minutos.}$

Por que “≈” e não “=”? Por arredondamento (43,2 virou “43”) e porque o mês é uma convenção — fevereiro tem 28 dias, outros meses têm 31. Por isso times de SRE costumam usar uma janela fixa (ex.: 28 dias) em vez do “mês corrente”.

Quem Decide a Meta (SLO)?

O error budget não é escolhido sozinho — ele nasce de uma cadeia de três decisões, do que se mede até o que se pode errar:



E o SLA (Service Level Agreement (Acordo de Nível de Serviço)?

É o compromisso formal (muitas vezes contratual) com o cliente — costuma ser mais frouxo que o SLO interno, para dar margem de segurança à equipe caso algo saia do controle.

Como Se Usa o Error Budget no Dia a Dia?

O error budget vira uma regra objetiva de decisão — em vez de discutir caso a caso se “dá para lançar” algo novo:

Orçamento disponível

- A equipe pode lançar novidades
- Pode assumir riscos calculados
- Experimentação é bem-vinda

Orçamento esgotado

- Lançamentos não essenciais são pausados
- Prioridade vira estabilizar o sistema
- Foco total em restaurar a confiabilidade

A vantagem: ninguém precisa “brigar” para decidir se lança ou não — o número já decide.

Um Exemplo Numérico

Um sistema tem SLO de 99,9% de disponibilidade. Num mês de 30 dias (43.200 minutos), o error budget é de 43,2 minutos de indisponibilidade.

30 min já consumidos (incidentes do mês)

13,2 min restantes

Restam 13,2 minutos de margem até o fim do mês.

Isso não significa pânico — mas é um sinal para a equipe: novos lançamentos arriscados devem esperar, e a prioridade passa a ser não usar o resto do orçamento até o próximo ciclo.

Da Discussão Subjetiva ao Número

- Sem *error budget*, a decisão “parar para corrigir confiabilidade” vira uma discussão política — quem grita mais alto, vence.
- Com *error budget*, a decisão é automática: orçamento zerado, prioridade muda. Sem debate.
- É a mesma lógica dos *quality gates* da Aula 5 — tirar a subjetividade de decisões de qualidade sempre que possível.

Conclusão

- As metas pragmáticas de SQA (requisitos, projeto, código, controle) orientam o que medir
- Pareto e Seis Sigma transformam dados de erro em prioridades de correção
- Confiabilidade e disponibilidade (MTBF, MTTF, MTTR) hoje vivem no vocabulário de SRE — SLI, SLO, error budget
- O plano de SQA é o roteiro que amarra tudo isso num projeto real

Fim da Unidade I

Encerramos aqui a Unidade I — Fundamentos de Engenharia de Software e Garantia da Qualidade. A próxima aula será de avaliação, sendo assim, preparem-se.

OBRIGADA PELA ATENÇÃO

Questionário Gamificado — Garantia de Qualidade de Software (Parte II)

QUESTIONÁRIO GAMIFICADO

Kahoot (AC1)